

JobFlow: Volcano作业编排引擎

目录

01 背景

02 Joblow介绍

03 Joblow工作原理

04 总结



01

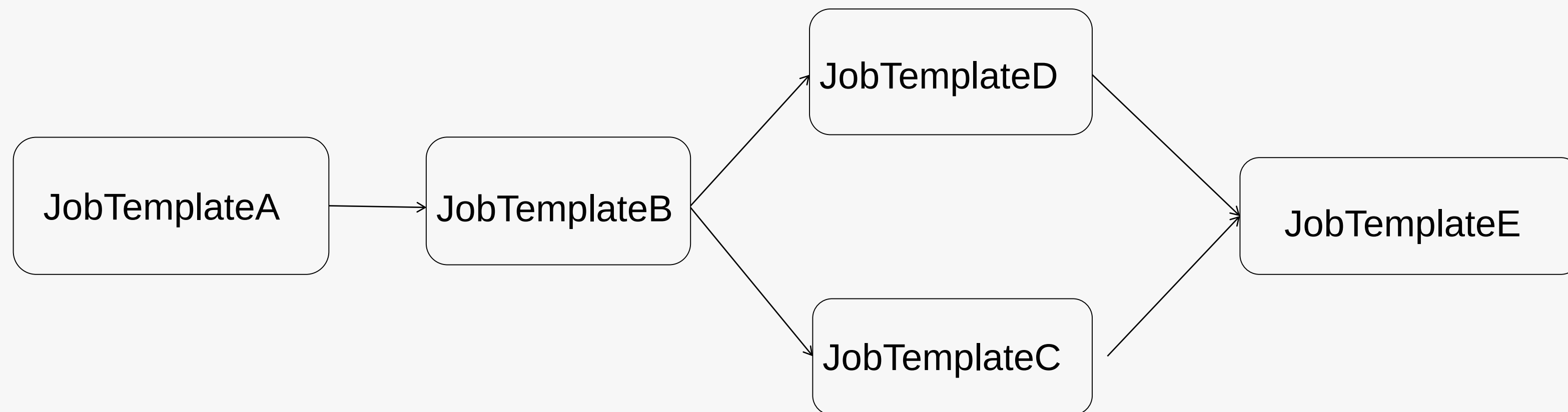
背景



JobFlow背景

Volcano的原生能力可以通过vcjob来实现。但是**vcjob目前没有提供多任务编排的能力，不能解决多任务依赖和数据交互的问题。**

如果使用vcjob，不太优雅的方式是每个任务轮询前一个任务的状态，直到结束。或者通过initcontainer，因为initcontainer是依次执行的，可以把前两个任务放在initcontainer中，把最后一个单元测试放在vcjob主模板中，这样就可以保证vcjob启动时，前一个initcontainer任务能够成功执行。但是，initcontainer只能解决依次执行的非常简单的串行多任务问题，而不能解决一些复杂的非线性任务调度。这些任务的依赖关系往往形成一个复杂的DAG（有向无环图），如：



JobFlow背景

目前，volcano并没有实现作业协调。目前的主要解决方案如下：

1. 手动提交多个作业

这种方式操作繁琐，重复操作太多，时间成本比较高。

2. 与现有操作平台相结合

采用企业自主研发的操作平台，满足自身需求

3. 使用Argo Workflow

Workflow提供了STEP和DAG两种方法来处理依赖结构的关系。这两种方法几乎涵盖了所有复杂的依赖结构，可以支持任务的串行并行。但是，在依赖关系类型方面，只提供任务完成、失败、错误等一般结果。

当用户用vcjob作为载体安排tasks时，他可能需要知道tasks的运行状态，也可能是用探针的形式探测任务的状态。

Workflow可以查看vcjob级别的进度（完成和未完成），但不能直观地显示vcjob下的任务和pod的完成情况，以及相应的状态信息。

基于以上背景，我们需要一个更轻量级的协调引擎，专注于并与Volcano一起工作。



02

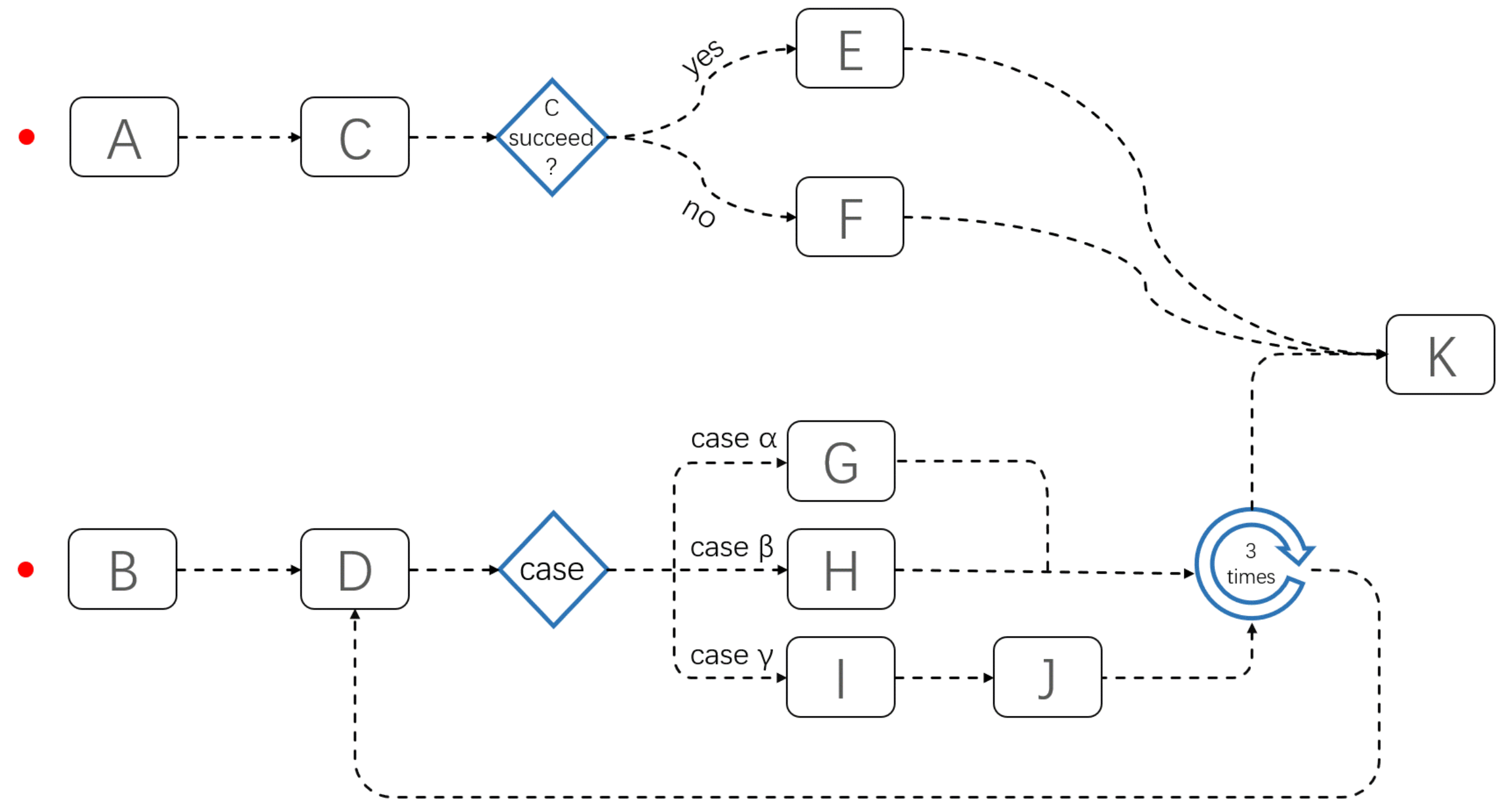
JobFlow介绍



JobFlow

对于使用vcjob场景下的任务调度，用户可以查看vcjob的完成进度和vcjob下的pod的完成进度。JobFlow可以根据vcjob添加相应任务的状态信息和执行进度，进而查看每个vcjob的相关状态等。

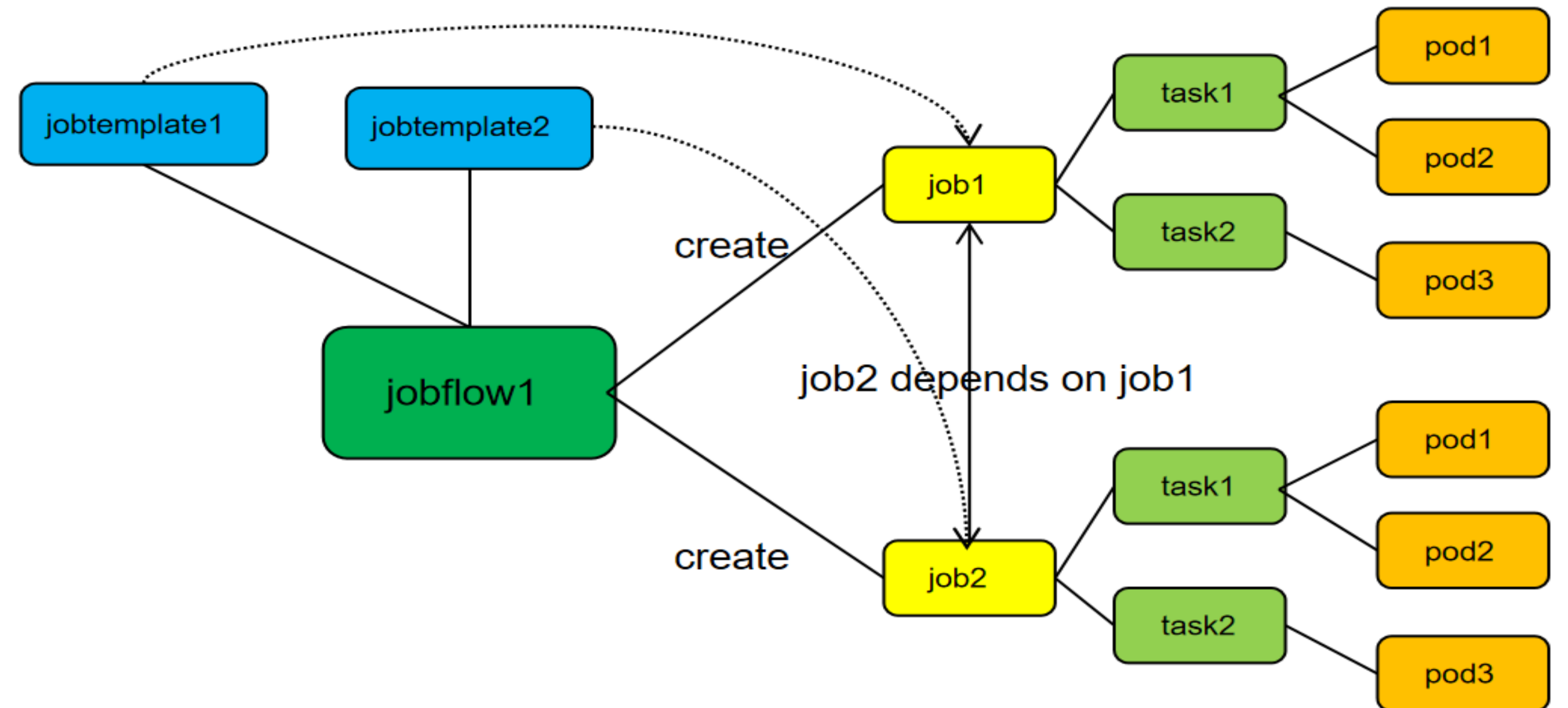
1. 当任务依赖关系处于完成状态时，JobFlow可以解决与Gang算法相矛盾的问题。
2. 提供更多样化的依赖场景，完成依赖，运行时依赖，探测等。
3. JobFlow与Volcano相关，但作为vcjob的上层资源，不会影响Volcano相关功能。



JobFlow和JobTemplate

JobFlow的完整操作包括两个CRD:

1. JobFlow: 用于描述vcjob之间的依赖关系。将vcjob的状态与JobFlow同步。控制器将根据JobFlow执行相关的处理。
2. Jobtemplate: jobtemplate是一个模板, 用于描述vcjob的各种信息。jobTemplate的规格直接使用vcjob的规格。一个jobtemplate的状态是用来跟踪基于该jobtemplate模板创建的vcjob。

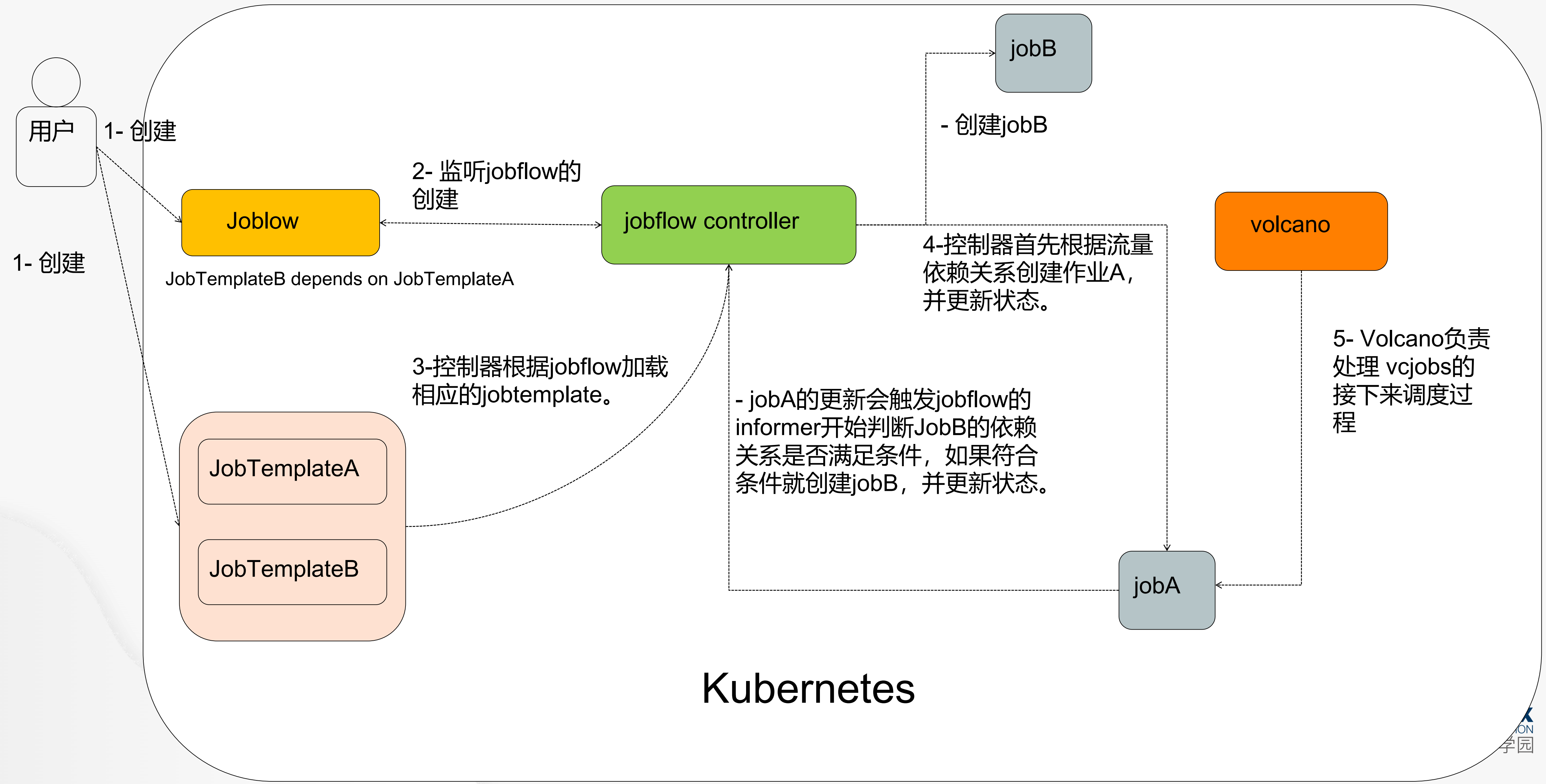


03

JobFlow工作原理



JobFlow工作原理

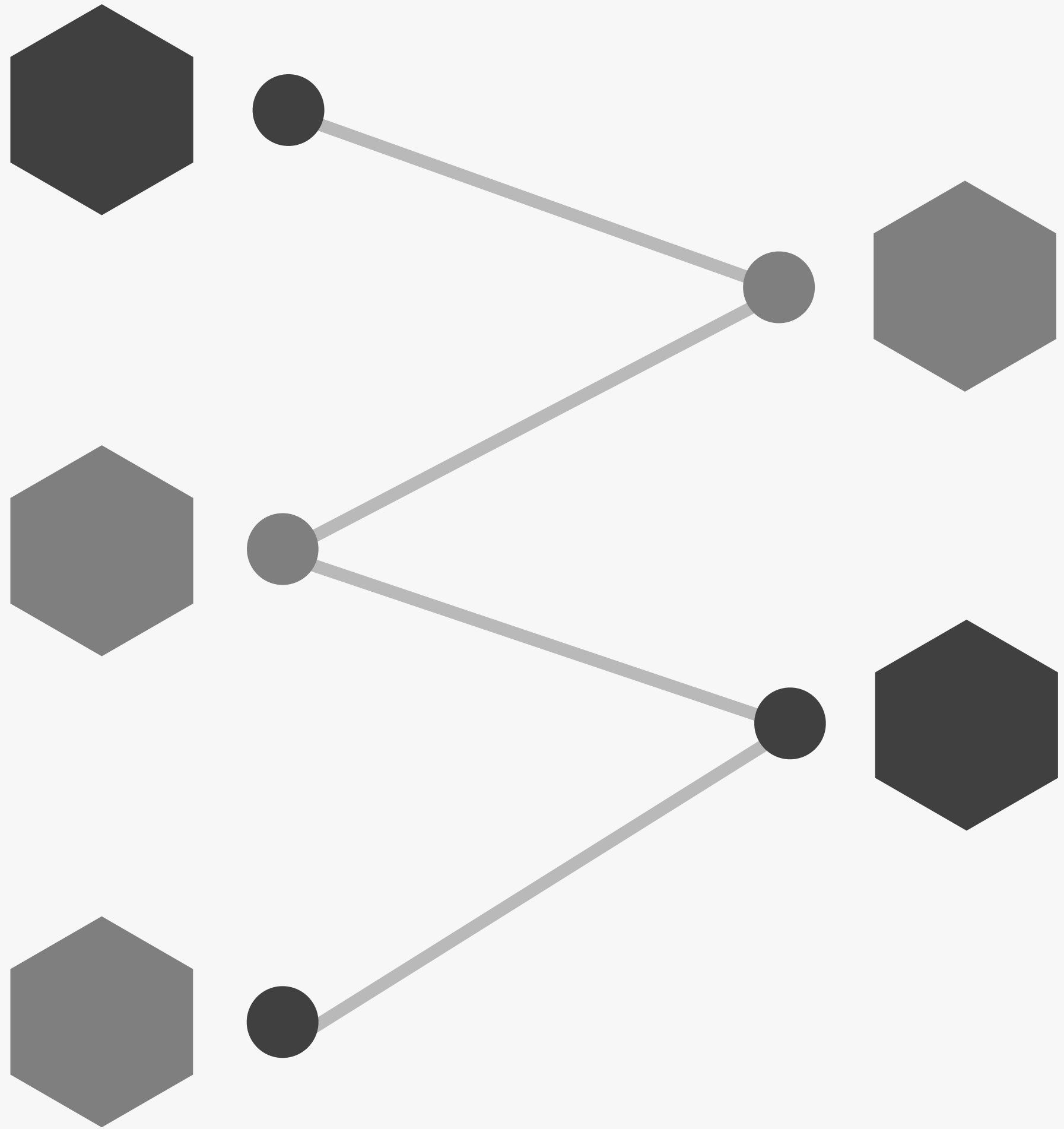


JobFlow工作原理

根据命名空间加载Jobflow的CRD。

遍历以确定剩余vcjob的依赖关系是否符合条件。如果符合条件，则创建剩余的vcjob。

为vcjob的更新添加informer。当vcjob状态改变时，同步jobflow的状态。



创建起始vcjob。

将当前状态信息写入Joblow的状态中。

JobFlow Spec

```
// JobFlowSpec defines the desired state of JobFlow
type JobFlowSpec struct {
    Flows          []Flow `json:"flows,omitempty"//retain/delete  jobflow执行完成后是否保留生成的job。
    JobRetainPolicy string `json:"jobRetainPolicy,omitempty"`
}
```

```
// Flow defines the dependent of jobs
type Flow struct {
    Name          string `json:"name"//JobTemplate 名称
    DependsOn     *DependsOn `json:"dependsOn,omitempty"`
}
```

```
type DependsOn struct {
    Targets []string `json:"targets,omitempty"//该JobTemplate所依赖的JobTemplate（默认为complete）。
    Probe   *Probe `json:"probe,omitempty"//当该字段为空时，默认为complete依赖。当配置后可以基于该字段来扩展
}
```

Jobflow的规范目前用flow和jobretainpolicy字段来描述vcjob的复杂依赖关系。

目前，Jobflow的规范只实现了完成状态的依赖关系，还没有实现其他的依赖关系类型。



JobFlow Status

```
// JobFlowStatus defines the observed state of JobFlow
type JobFlowStatus struct {
    PendingJobs    []string `json:"pendingJobs,omitempty"`
    RunningJobs    []string `json:"runningJobs,omitempty"`
    FailedJobs     []string `json:"failedJobs,omitempty"`
    CompletedJobs  []string `json:"completedJobs,omitempty"`
    TerminatedJobs []string `json:"terminatedJobs,omitempty"`
    UnKnowJobs     []string `json:"unKnowJobs,omitempty"`
    JobStatusList  []JobStatus `json:"jobStatusList,omitempty"`
    Conditions     map[string]Condition `json:"conditions,omitempty"`
    State          State `json:"state,omitempty"`
}
```

```
type Condition struct {
    Phase          v1alpha1.JobPhase `json:"phase,omitempty"`
    CreateTimestamp metav1.Time `json:"createTime,omitempty"`
    RunningDuration *metav1.Duration `json:"runningDuration,omitempty"`
    TaskStatusCount map[string]v1alpha1.TaskState `json:"taskStatusCount,omitempty"`
}
```

Jobflow的状态描述了jobflow中所有vcjob的当前状态，相应vcjob的创建时间，以及任务中的工作状态信息。

JobFlow Status

Jobflow状态描述了工作流中所有VCJob的当前状态，相应VCjob的创建时间，以及相应vcjob的工作状态。

```
type JobStatus struct {  
    Name            string            `json:"name,omitempty"`  
    State           v1alpha1.JobPhase `json:"state,omitempty"`  
    StartTimestamp  metav1.Time        `json:"startTimestamp,omitempty"`  
    EndTimestamp    metav1.Time        `json:"endTimestamp,omitempty"`  
    RestartCount    int32              `json:"restartCount,omitempty"`  
    RunningHistories []JobRunningHistory `json:"runningHistories,omitempty"`  
}
```

```
type JobRunningHistory struct {  
    StartTimestamp metav1.Time        `json:"startTimestamp,omitempty"`  
    EndTimestamp   metav1.Time        `json:"endTimestamp,omitempty"`  
    State          v1alpha1.JobPhase `json:"state,omitempty"`  
}
```

04

总结



JobFlow and the future

JobFlow是为了解决Volcano本身不提供作业流编排的问题。与其他工作流编排引擎（如Argo Workflow）相比，Argo Workflow服务于通用作业。存在着无法获得Volcano作业的各种状态，以及无法与Volcano合作以实现后期更具体的要求的缺点。作为Volcano的一个子项目，JobFlow对Volcano功能有更多的扩展性，未来将提供更多的作业依赖类型，更多的作业执行策略等等。

未来计划

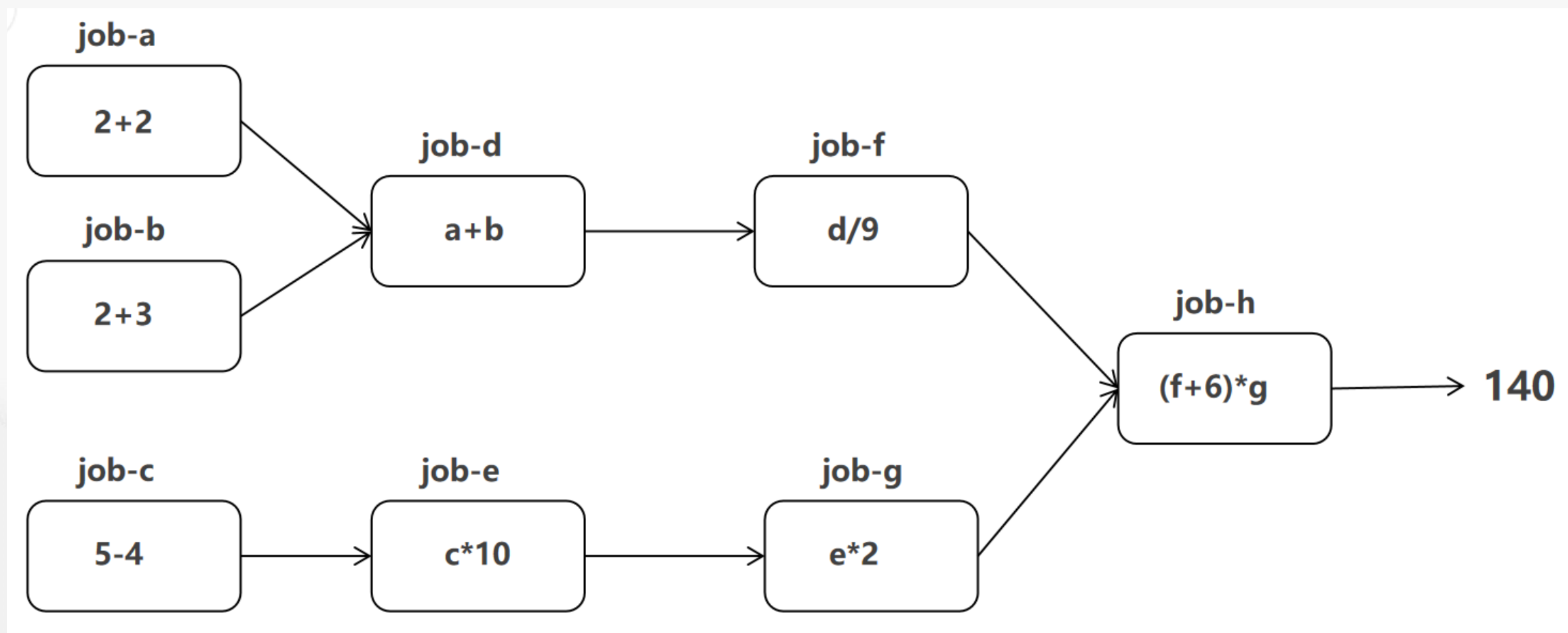
- JobFlow将提供更多的依赖和判断方式。实现条件性和循环等依赖关系。提供基于计算结果的条件性依赖。
- JobFlow还没有与Volcano有太多的互动。在未来，与Volcano的协调有望提高Volcano的认知度。当然，这部分还有待研究。



JobFlow Progress

目前, JobFlow已经成为volcano的一个子项目, JobFlow暂时只实现了基本的依赖关系, 同步了volcano vcjob的相关状态等。未来需要完善更全面的功能。

JobFlow cases



Demo

https://www.bilibili.com/video/BV1c44y1Yspm_id_from333..0.0



Join Volcano



Website: <https://volcano.sh/en/>



Github: <https://github.com/volcano-sh/volcano>



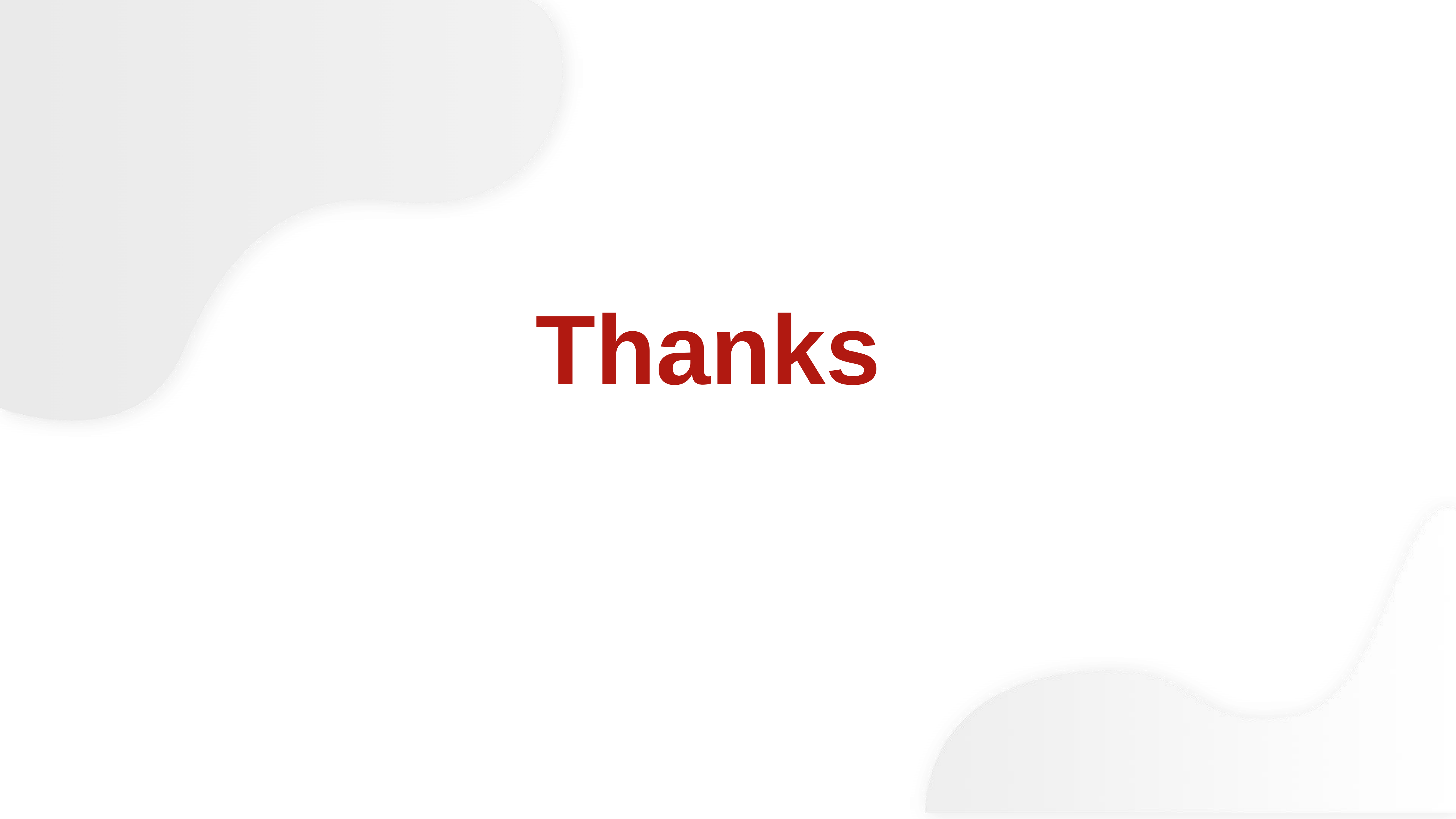
Slack Channel: <https://volcano-sh.slack.com/>



Follow Us



Join Group



Thanks